A HANDS-ON WORKSHOP

# Beyond AI Slop

## Building better interfaces with design systems

Junead Khan · Founder, Treasury

START HERE

# Join the studio. Pair up if needed.

`juneadkhan.github.io/hacknc-beyond-ai-slop/`

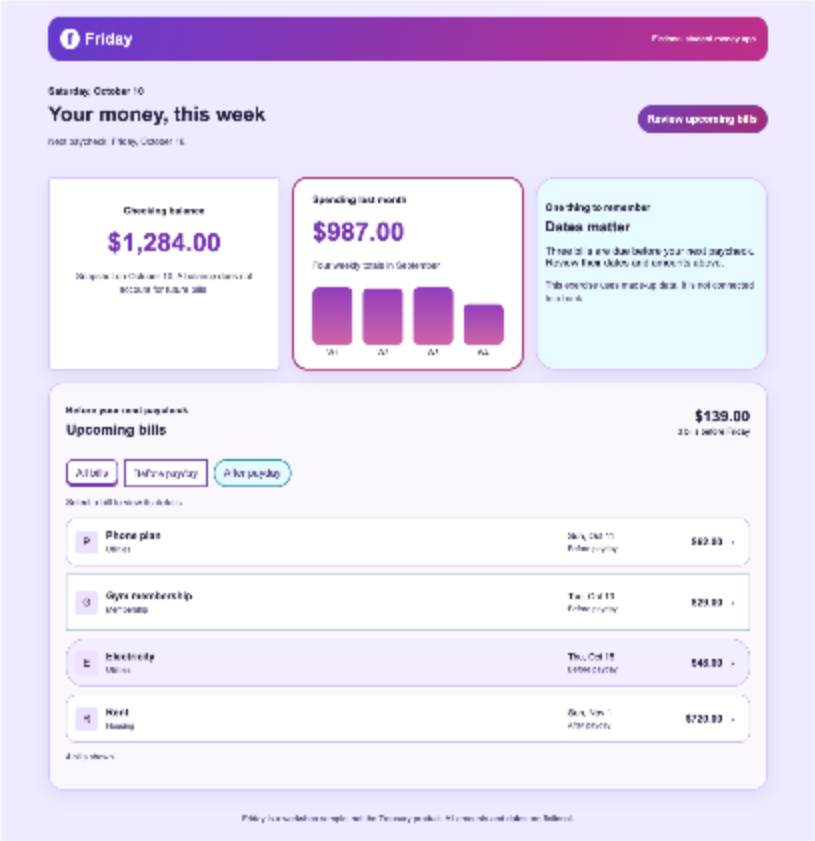


## No setup for the studio

Browser + your AI chat. Nothing to install.

No AI access? Use a reference
and change its tokens.

# First, try the user's task.




## Which bills are due before Friday?

Find the dates and amounts.

Same task for every style.

# Name the failure you can observe.

## Hierarchy

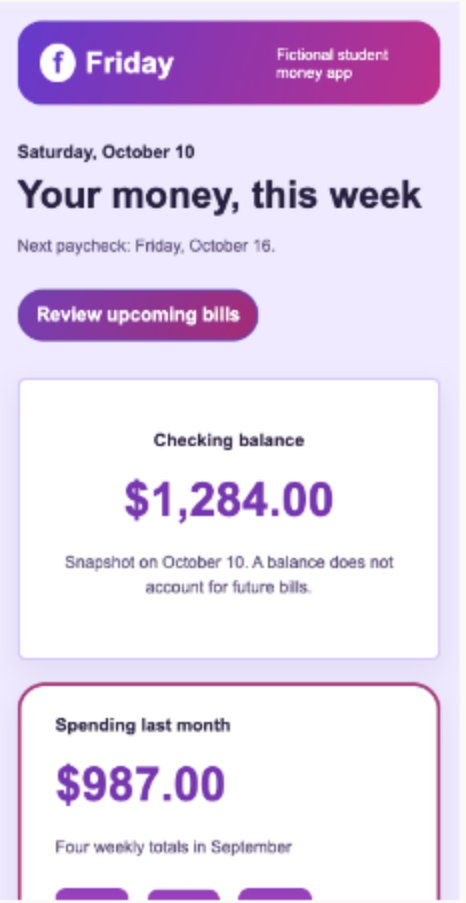
The secondary metrics get attention before bill dates.

## Component rules

Three filters look like three unrelated controls.

## Small screens

The important list moves below secondary content.



# Give AI decisions it can reuse.

01

## Purpose

Find bills before payday.

02

## Tokens

Name roles, then assign values.

03

## Components

Reuse rows, filters and panels.

04

## Rules

Define priority, states and mobile.

A shared system still leaves room for your own taste.

# Name the role. Reuse the value.

```
:root {
  --surface: #ffffff;
  --text: #1c303c;
  --accent: #214e63;
  --on-accent: #ffffff;
  --space-4: 16px;
}
```

```
.button-primary {
  background: var(--accent);
  color: var(--on-accent);
  padding: var(--space-4);
}
```

One shared decision can update several components.

# Specify the states and the exceptions.

## Bill rows

Same date, amount and spacing rules for every bill.

## Buttons and filters

Default, hover, selected and visible keyboard focus.

## Responsive hierarchy

Bills remain first; amounts stay beside their own bill.

# Start with foundations that fit your project.

## Carbon

IBM’s product system

Foundations, components and usage guidance.

## Fluent 2

Microsoft’s design system

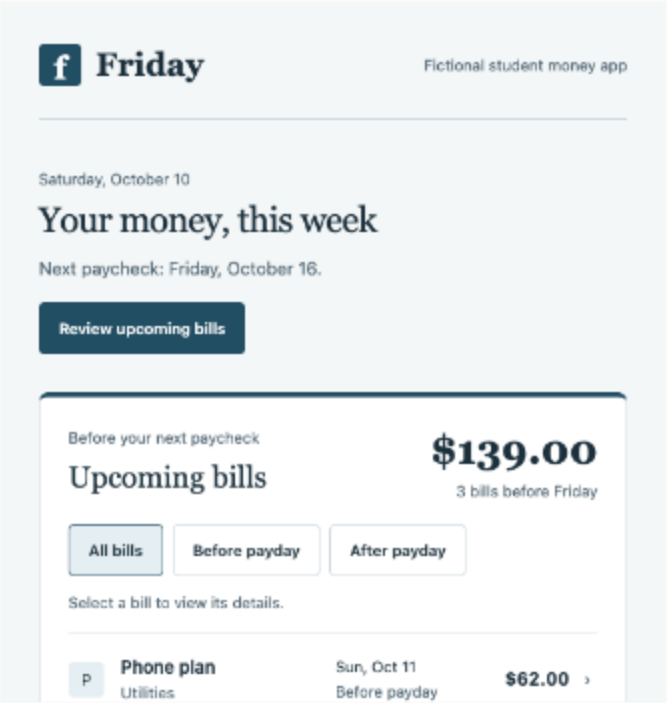
Resources and components for several platforms.

## Radix Primitives

Unstyled behaviors

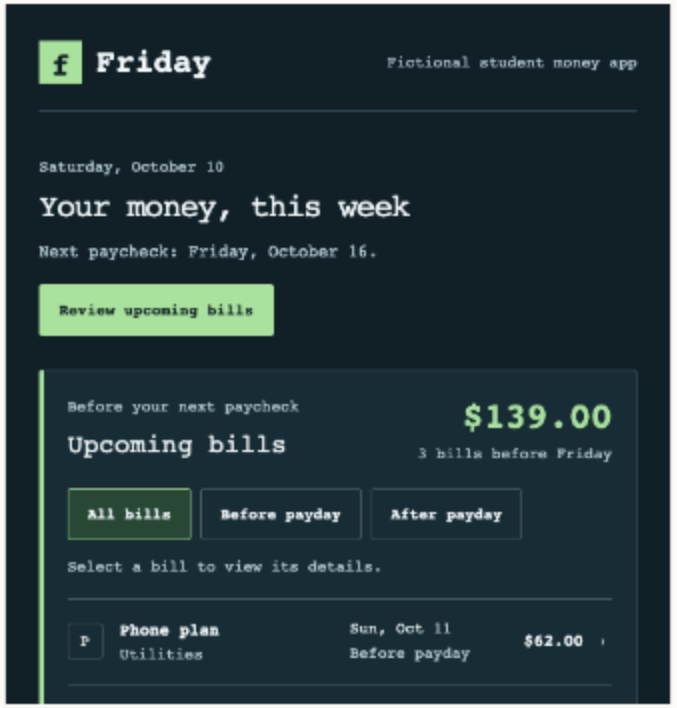
Accessible building blocks; you supply visual rules.

# Different personalities. Shared discipline.



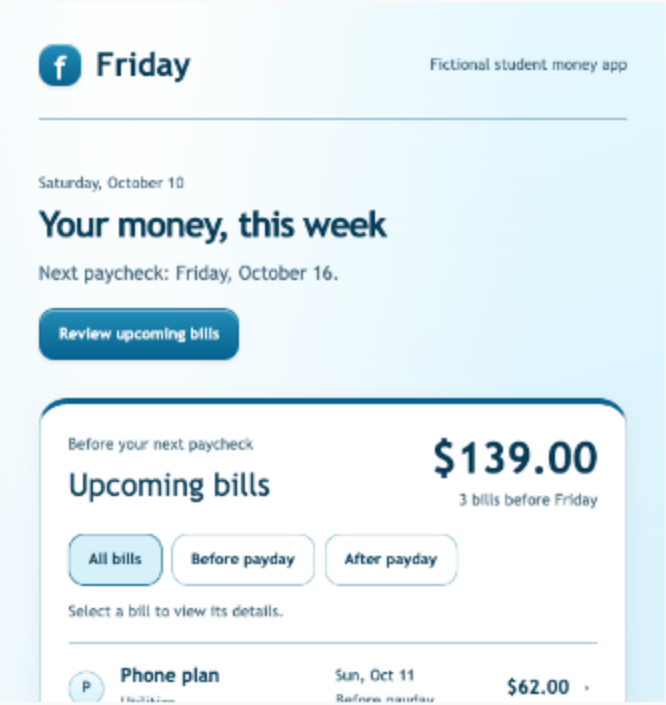

## Ledger

Quiet reading




## Signal

Technical clarity




## Aero

Optimistic gloss

# Guide first. Implementation second.

## 1 Create and review the guide

Purpose, tokens, component rules, states and mobile.

## 2 Apply one complete stylesheet

Use the fixed DOM and facts. Inspect the rendered app.

## 3 Revise a specific observed failure

“At 390px, the date wraps away from its amount.”

# Your style. The same working app.

## Choose a point of view

Audience + mood + typography + density + priority.

## Generate → review → apply

A design guide, then a complete CSS stylesheet.

## Check mobile and keyboard

Keep every amount, date, filter and detail dialog.

12

minutes

No AI access?
Use a reference
and change tokens.

# Swap laptops. Let the screen explain.

## Give your partner the task

“Find every bill due before Friday. Open one.”

## Look for evidence

Dates and amounts, shared rules, 390px, keyboard.

## Share one rule and one remaining issue

What improved? What still gets in the user’s way?

# Bring the rules into your own hack.

## Write the task and save the guide

Give your coding agent rules it can keep returning to.

## Reuse components; change one thing at a time

Inspect the rendered result before moving on.

## Make trustworthy information easy to use

A clear date and amount matter more than decoration.

# Keep your work. Keep asking why.

## Download your app + your design guide

Keep the working interface and the decisions behind it.



## Which rule will you use in your hack?

Junead Khan · treasury.sh